

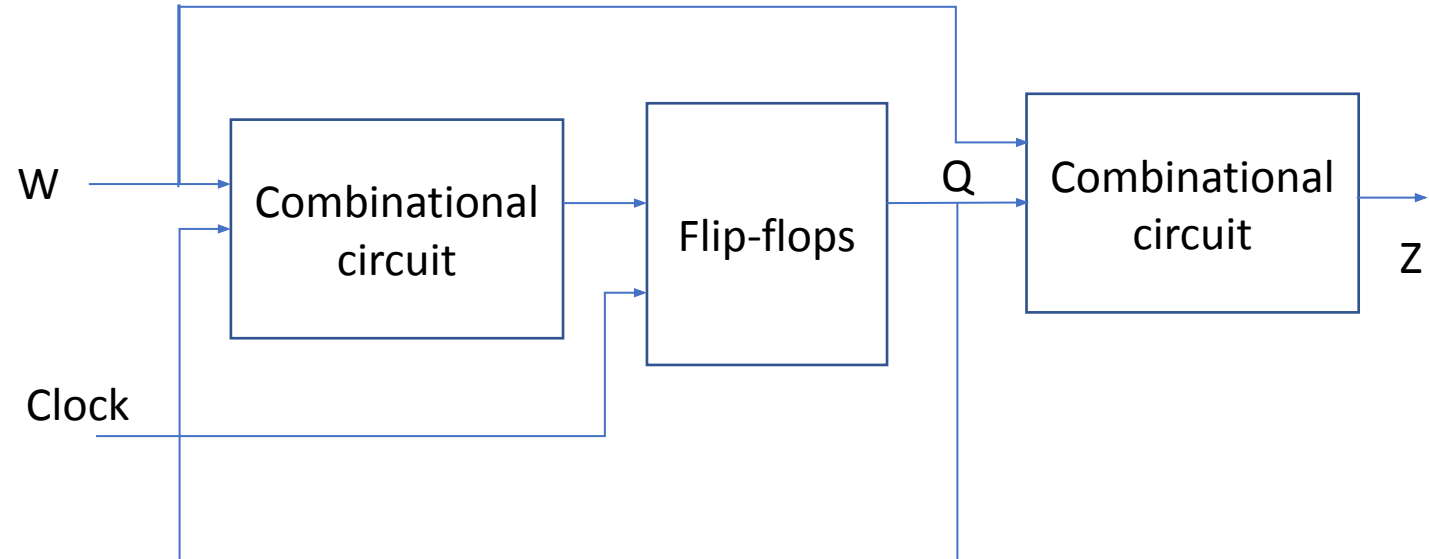
# CSE 460: VLSI Design (Lab)

## Experiment 4

Simulation of Mealy and Moore Type FSMs in Verilog

# Finite State Machine

- In Combinational logic circuits, outputs are determined solely by the present values of the inputs.
- In Sequential circuits, outputs depend both on the past behavior of the circuit and the present values of the inputs. The storage elements in terms of flip-flops can reserve the state information of the logic circuit at any given time.
- The sequential circuits in which a clock signal is used to control the operation of the circuit, is called synchronous sequential circuits. These synchronous sequential circuits are known in general as Finite State Machines (FSMs).
- The building blocks of a finite state machine are combinational circuits and one or more flip-flops. Under the control of the clock signal, the flip-flops change their state as determined by the combinational logic.



# Finite State Machine

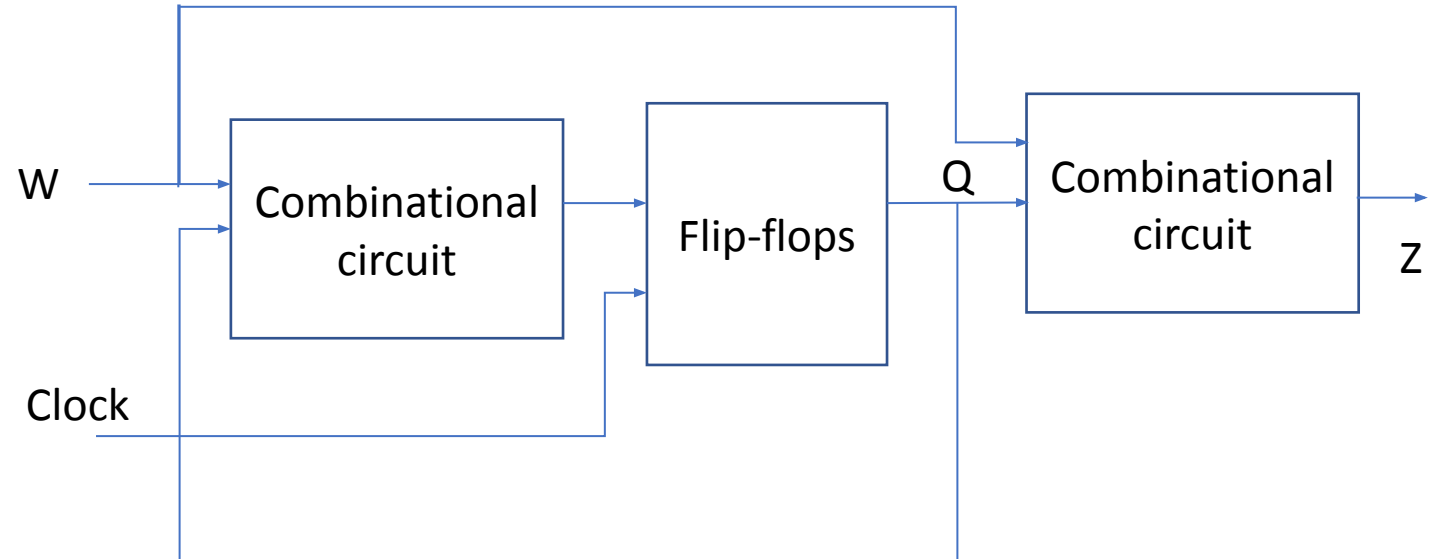
FSMs can be of two types:

## 1. Moore type FSM

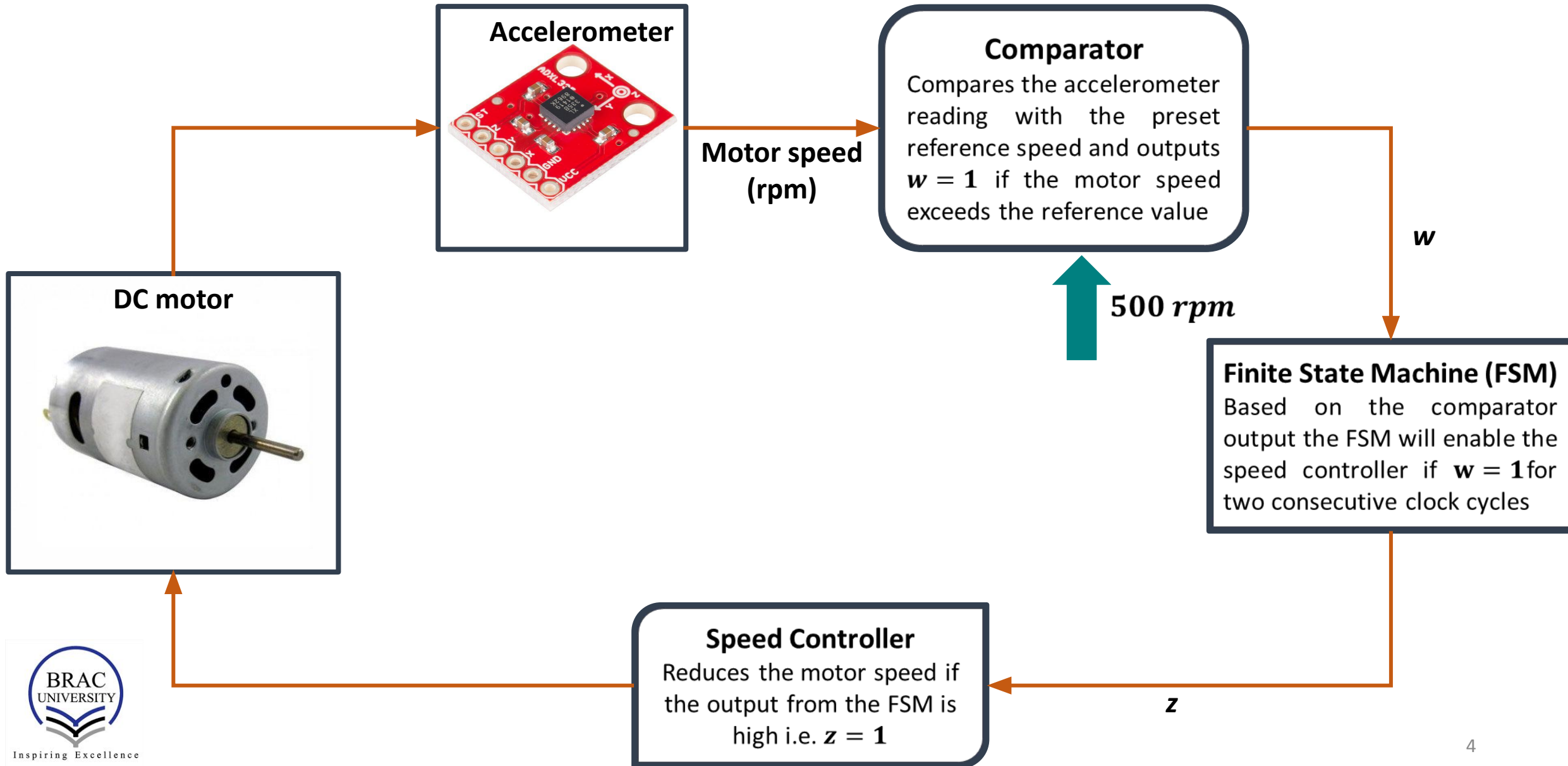
The sequential circuits whose outputs depend **only on the states** of the circuit are of Moore type.

## 2. Mealy type FSM

The sequential circuits whose outputs depend **on both the state and the present primary inputs** are of Mealy type



# FSM based motor speed controller



# Moore Type FSM

## Problem:

Suppose that we wish to design a circuit that meets the following specification:

- The circuit has one input, **w**, and one output, **z**.
- All changes in the circuit occur on the positive edge of a clock signal.
- The output **z** is equal to **1** if during **two immediately preceding** clock cycles the input **w** was equal to 1. Otherwise, the value of **z** is equal to 0.

## Input-output combination:

| Clock Cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-------------|---|---|---|---|---|---|---|---|---|----|
| w           | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0  |
| z           | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1  |

# Moore Type FSM

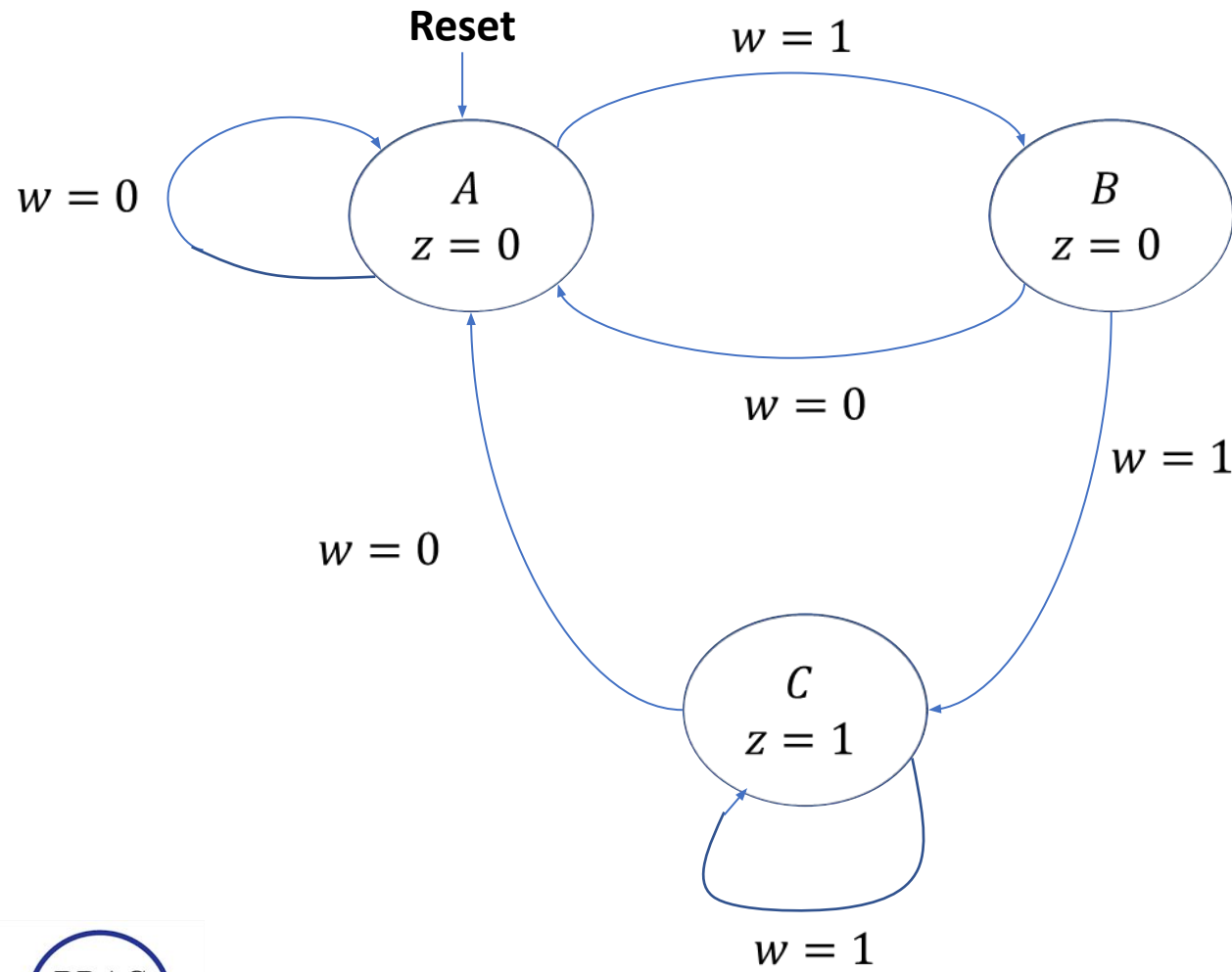


Figure : State Diagram

| Present State | Next State |     | Output z |
|---------------|------------|-----|----------|
|               | w=0        | w=1 |          |
| A             | A          | B   | 0        |
| B             | A          | C   | 0        |
| C             | A          | C   | 1        |

Figure : State Table

# Moore Type FSM

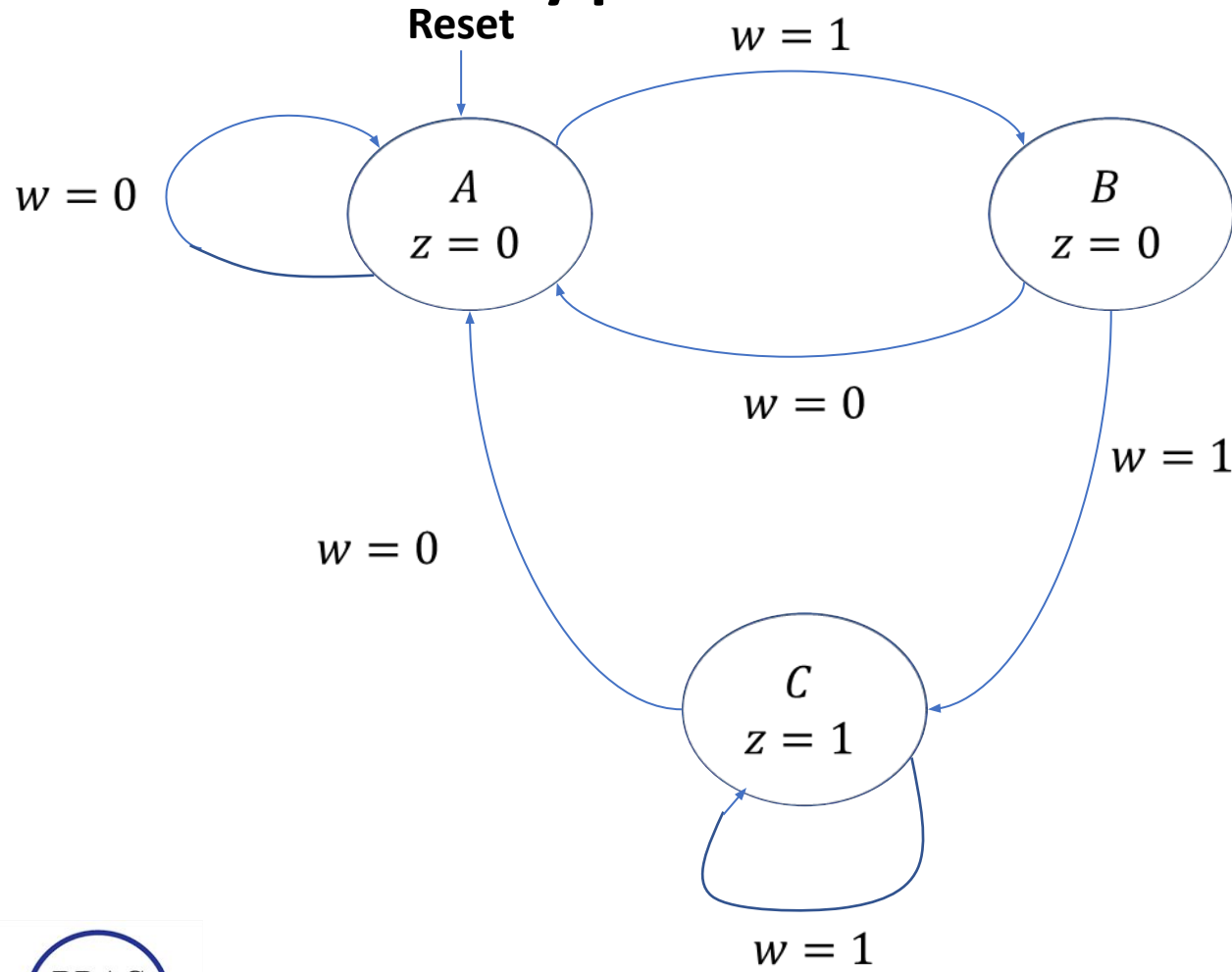
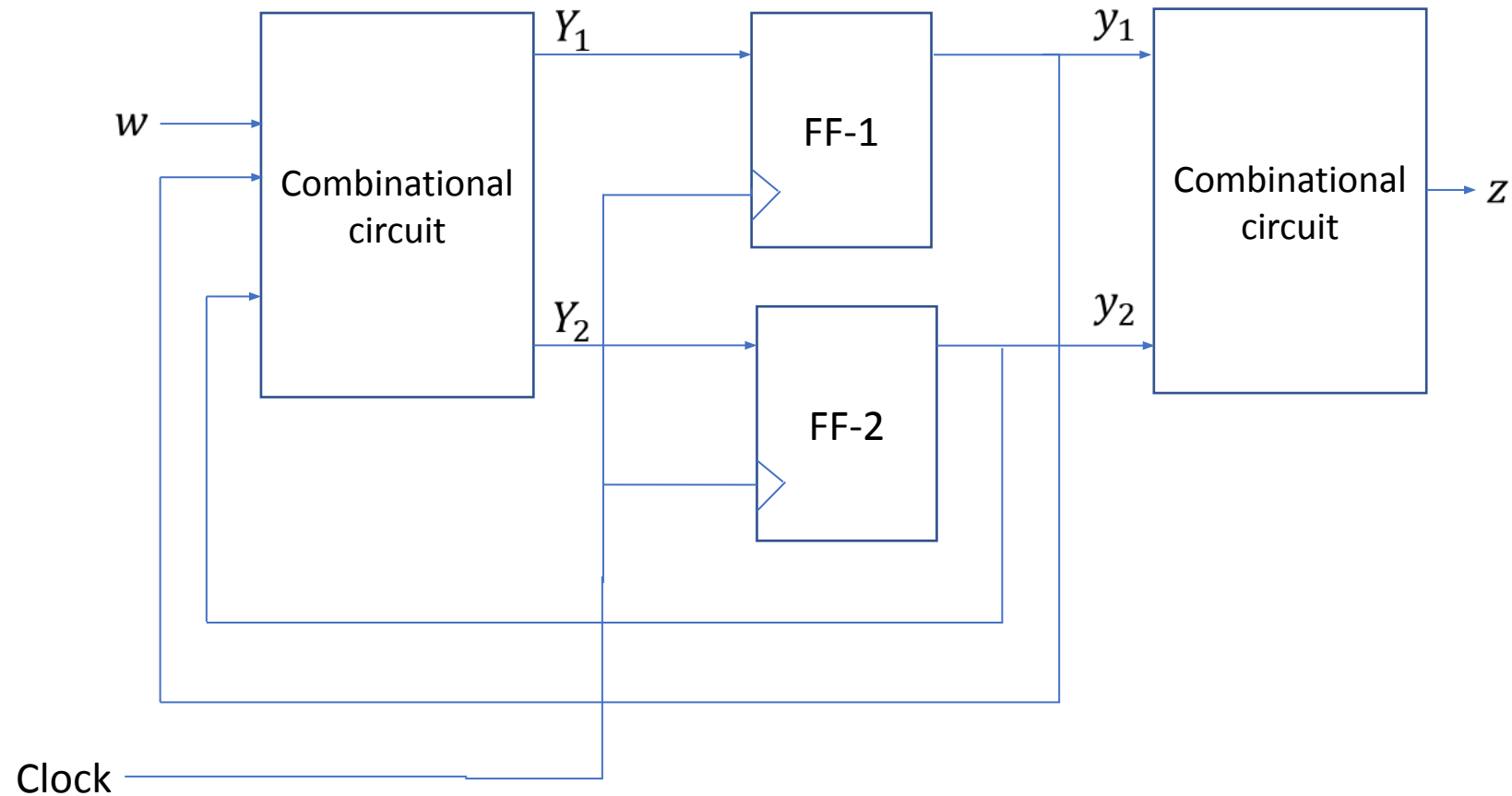


Figure : State Diagram

|       | Next State |       | Output<br>z |
|-------|------------|-------|-------------|
|       | w=0        | w=1   |             |
| 00(A) | 00(A)      | 01(B) | 0           |
| 01(B) | 00(A)      | 10(C) | 0           |
| 10(C) | 00(A)      | 10(C) | 1           |
| 11    | dd         | dd    | d           |

# General Circuit Diagram of the FSM

|    | Next State |     | Output<br>z |
|----|------------|-----|-------------|
|    | w=0        | w=1 |             |
| 00 | 00         | 01  | 0           |
| 01 | 00         | 10  | 0           |
| 10 | 00         | 10  | 1           |
| 11 | dd         | dd  | d           |





# Designing the FSM in Verilog

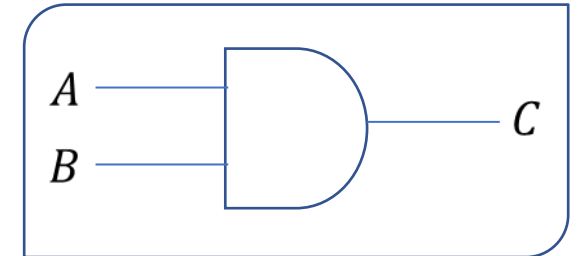
In order to design the FSM using Verilog we will need to perform the following steps:

- Assigning binary variables to states
- A mechanism for figuring out the current state (Sequential circuit block)
- Defining the logic for state transition (Combinational circuit block)
- Defining the output for the current state (Continuous assignment statement)

## Combinational circuit synthesis in Verilog

`always@ (*)` blocks are used to describe combinational logic in Verilog. **Blocking assignment** statements are to be used inside these blocks.

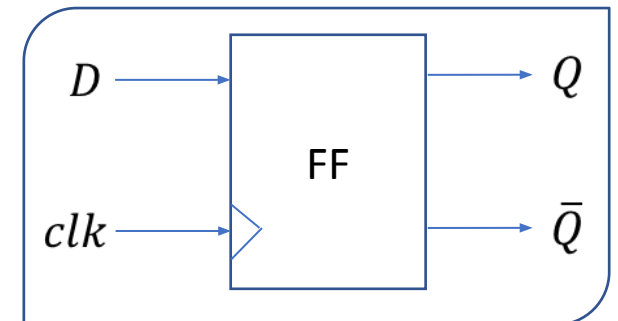
```
always@ (*)  
begin  
    C = A & B;  
end
```



## Sequential circuit synthesis in Verilog

`always@ (posedge clk)` or `always@ (negedge clk)` blocks are used to describe sequential logic in Verilog. **Non-blocking assignment** statements are to be used inside these blocks.

```
always@ (posedge clk)  
begin  
    Q <= D;  
end
```



# Moore Type FSM

|       | Next State |       | Output<br>z |
|-------|------------|-------|-------------|
|       | w=0        | w=1   |             |
| 00(A) | 00(A)      | 01(B) | 0           |
| 01(B) | 00(A)      | 10(C) | 0           |
| 10(C) | 00(A)      | 10(C) | 1           |
| 11    | dd         | dd    | d           |

module Moore11 (Clock, Resetn, w, z,y,Y);

input Clock, Resetn, w;  
output reg z;  
output reg [2:1] y, Y;  
parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10;

// Define the next state combinational circuit  
always @(w, y)

case (y)  
  
A: if (w) Y = B;  
    else Y = A;  
B: if (w) Y = C;  
    else Y = A;  
C: if (w) Y = C;  
    else Y = A;  
default: Y = 2'bxx;

endcase

// Define the sequential block  
always @(negedge Resetn, posedge Clock) begin

if (Resetn == 0) y <= A;

else y <= Y;

end

// Define output

always@(y) begin

case (y)  
  
A: z=0;  
B: z=0;  
C: z=1;

endcase

end

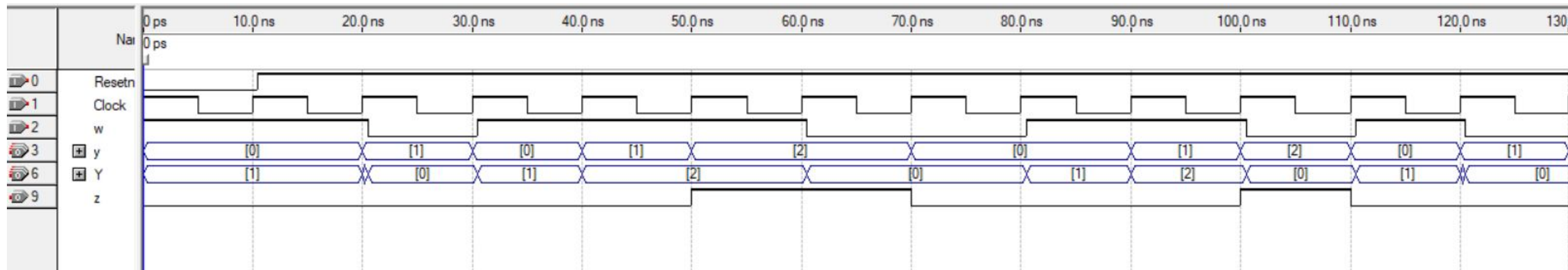
endmodule

# Moore Type FSM

Steps for properly inserting inputs (w as well as resetn):

1. first invert the Clock signal
2. Introduce now a small offset(0.5ns) in both w and Resetn signals

Video link : <https://youtu.be/ZdOeTZLUr0Q?feature=shared>



# Mealy Type FSM

## Problem:

We wish to design a circuit that meets the following specification:

- The circuit has one input, **w**, and one output, **z**.
- All changes in the circuit occur on the positive edge of a clock signal.
- The output **z** is equal to **1** if during **two immediate** clock cycles the input **w** was equal to 1. Otherwise, the value of **z** is equal to 0.

## Input-output combination:

| Clock Cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-------------|---|---|---|---|---|---|---|---|---|----|
| w           | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0  |
| z           | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0  |

# Mealy Type FSM

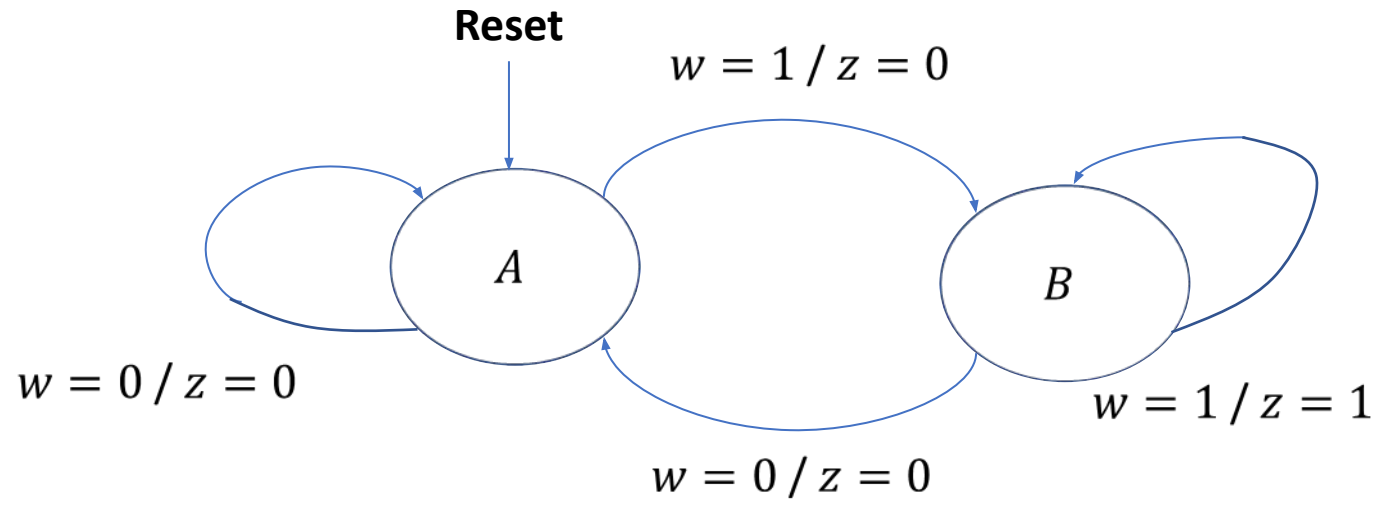


Figure : State Diagram

| Present State | Next State |     | Output z |     |
|---------------|------------|-----|----------|-----|
|               | w=0        | w=1 | w=0      | w=1 |
| A             | A          | B   | 0        | 0   |
| B             | A          | B   | 0        | 1   |

Figure : State Table

# Mealy Type FSM

| Present State | Next State |     | Output z |     |
|---------------|------------|-----|----------|-----|
|               | w=0        | w=1 | w=0      | w=1 |
| A             | A          | B   | 0        | 0   |
| B             | A          | B   | 0        | 1   |

Figure : State Table

| Present State<br>y | Next State |          | Output z |     |
|--------------------|------------|----------|----------|-----|
|                    | w=0<br>Y   | w=1<br>Y | w=0      | w=1 |
| 0(A)               | 0(A)       | 1(B)     | 0        | 0   |
| 1(B)               | 0(A)       | 1(B)     | 0        | 1   |

Figure : State-assigned Table

```
module Mealy11 (Clock, Resetn, w, z,y,Y);
```

```
    input Clock, Resetn, w;  
    output reg z;  
    output reg y, Y;  
    parameter A = 1'b0, B = 1'b1;
```

```
    // Define the next state and output "combinational" circuits
```

```
    always @(w, y)  
    case (y)
```

```
        A:      if (w)
```

```
            begin  
                z = 0;  
                Y = B;  
            end
```

```
            else  
            begin  
                z = 0;  
                Y = A;  
            end
```

```
        B:      if (w)
```

```
            begin  
                z = 1;  
                Y = B;  
            end
```

```
            else  
            begin  
                z = 0;  
                Y = A;  
            end
```

```
    endcase
```

```
    // Define the sequential block
```

```
    always @(negedge Resetn, posedge Clock) begin
```

```
        if (Resetn == 0) y <= A;
```

```
        else y <= Y;
```

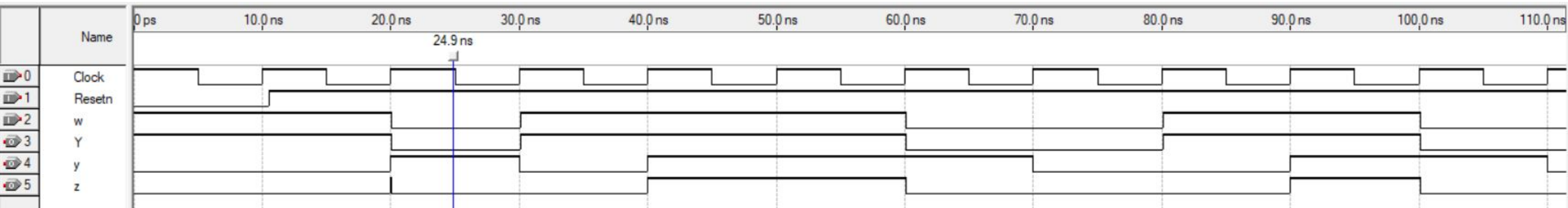
```
    end
```

```
endmodule
```



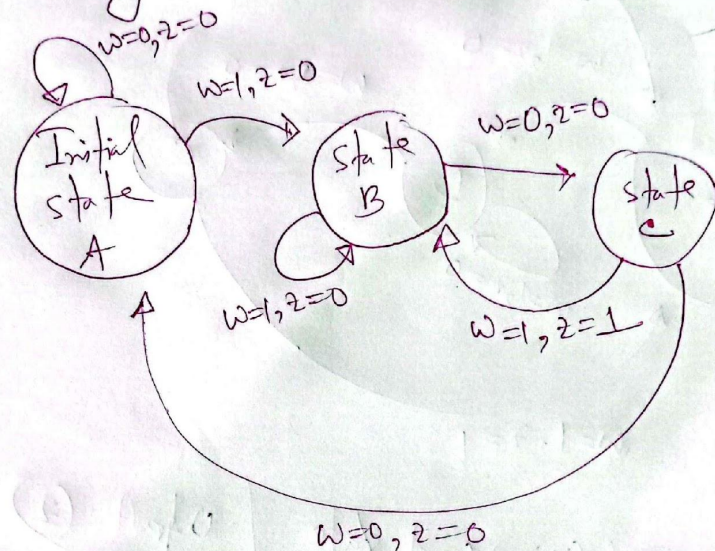
Steps for properly inserting inputs (w as well as resetn):

1. first invert the Clock signal
2. Introduce now a small offset(0.5ns) in both w and Resetn signals



## Mealy Type FSM

⊕ Detecting Sequence: "101" (overlapping)



| Present<br>state (x,y) | Next state |       | Output (z) |     |
|------------------------|------------|-------|------------|-----|
|                        | w=0        | w=1   | w=0        | w=1 |
| (00) A                 | A(00)      | B(01) | 0          | 0   |
| (01) B                 | C(10)      | B(01) | 0          | 0   |
| (10) C                 | A(00)      | B(01) | 0          | 1   |

```
module Mealy101(Clock, Resetn, w, z,y,Y);
```

```
input Clock, Resetn, w;
```

```
output reg z;
```

```
output reg [1:0]y, Y;
```

```
parameter A = 2'b00, B = 2'b01, C = 2'b10;
```

```
// Define the next state and output "combinational" circuits
```

```
always @(w, y)
```

```
case (y)
```

```
A:      if (w)
```

```
begin
  z = 0;
  Y = B;
end
```

```
else
begin
  z = 0;
  Y = A;
end
```

```
B:      if (w)
```

```
begin
  z = 0;
  Y = B;
end
```

```
else
begin
  z = 0;
  Y = C;
end
```

```
C:      if (w)
```

```
begin
  z = 1;
  Y = B;
end
```

```
else
begin
  z = 0;
  Y = A;
end
```

```
endcase
```

```
// Define the sequential block
```

```
always @(negedge Resetn, posedge Clock) // Asynchronous reset
```

```
begin
```

```
if (Resetn == 0) y <= A;
```

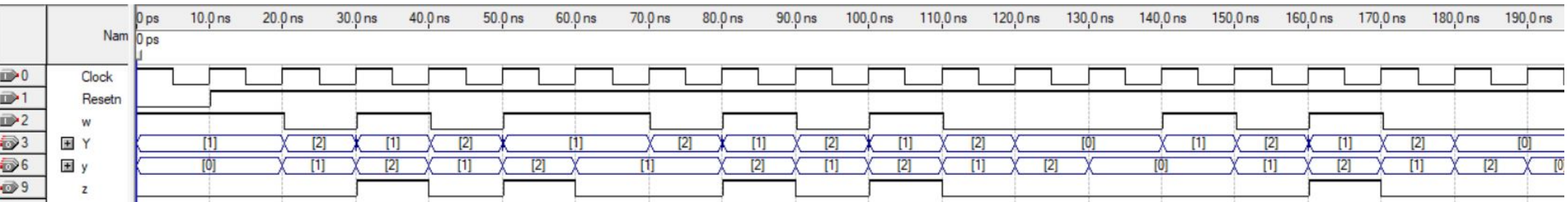
```
else y <= Y;
```

```
end
```

```
endmodule
```

Steps for properly inserting inputs (w as well as resetn):

1. first invert the Clock signal
2. Introduce now a small offset(0.5ns) in both w and Resetn signals



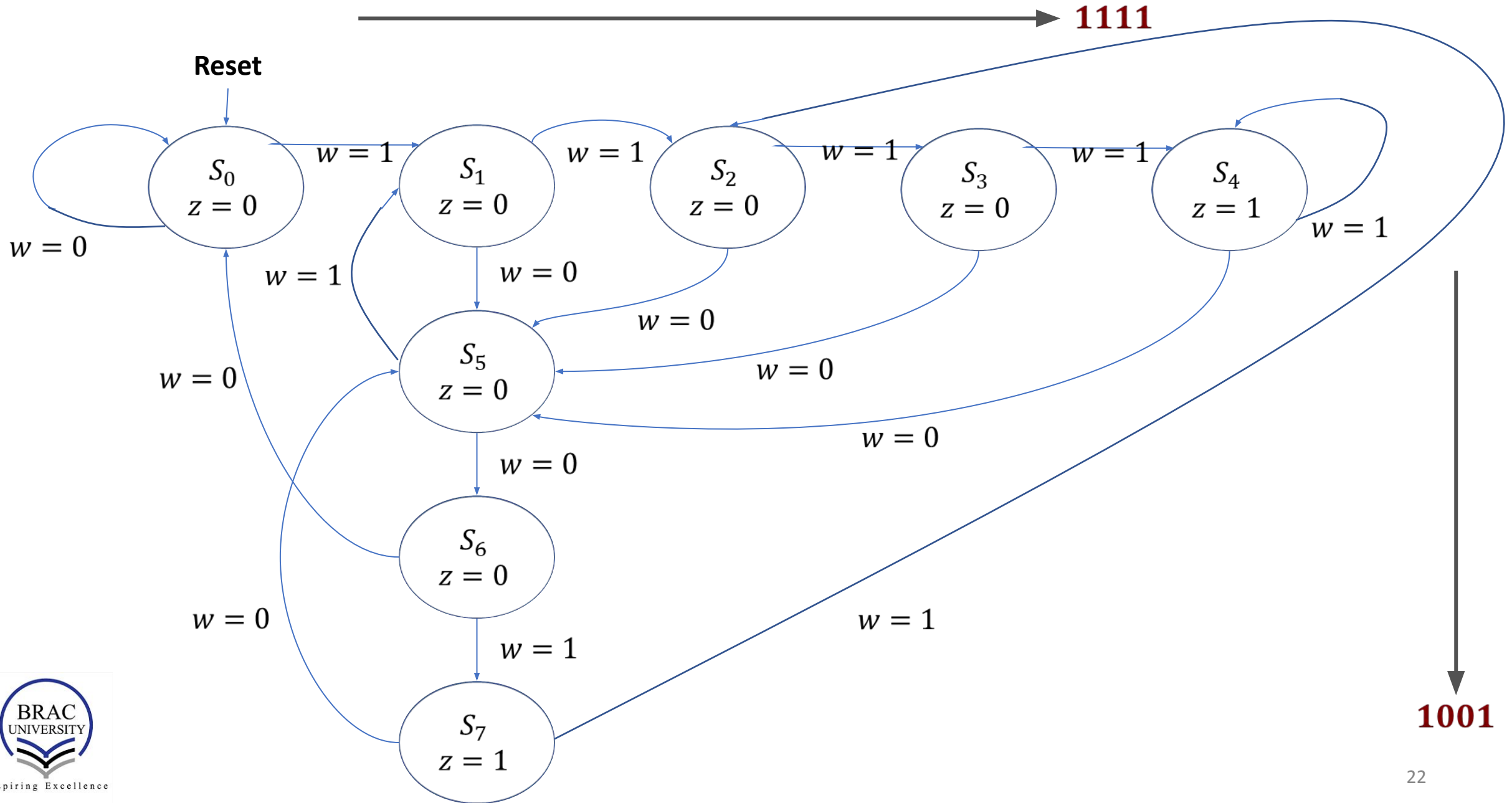
For Practice  
at home

# An Example (Moore type)

Problem: Derive the state diagram for an FSM that has an input  $w$  and an output  $z$ . The machine has to generate  $z = 1$  when the previous four values of  $w$  were 1001 or 1111; otherwise,  $z = 0$ . Overlapping input patterns are allowed. An example of the desired behavior is

| Clock cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|-------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|
| w           | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1  | 1  | 0  | 0  | 1  | 1  | 1  | 1  |
| z           | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0  | 1  | 0  | 0  | 0  | 1  | 0  | 0  |

# State diagram(Moore type)



# State table (Moore type)

| Present State | Next State |     | Output<br>z |
|---------------|------------|-----|-------------|
|               | w=0        | w=1 |             |
| S0            | S0         | S1  | 0           |
| S1            | S5         | S2  | 0           |
| S2            | S5         | S3  | 0           |
| S3            | S5         | S4  | 0           |
| S4            | S5         | S4  | 1           |
| S5            | S6         | S1  | 0           |
| S6            | S0         | S7  | 0           |
| S7            | S5         | S2  | 1           |

Figure : State Table

|         | Next State |         | Output<br>z |
|---------|------------|---------|-------------|
|         | w=0        | w=1     |             |
| 000(S0) | 000(S0)    | 001(S1) | 0           |
| 001(S1) | 101(S5)    | 010(S2) | 0           |
| 010(S2) | 101(S5)    | 011(S3) | 0           |
| 011(S3) | 101(S5)    | 100(S4) | 0           |
| 100(S4) | 101(S5)    | 100(S4) | 1           |
| 101(S5) | 110(S6)    | 001(S1) | 0           |
| 110(S6) | 000(S0)    | 111(S7) | 0           |
| 111(S7) | 101(S5)    | 010(S2) | 1           |

Figure : State-assigned Table

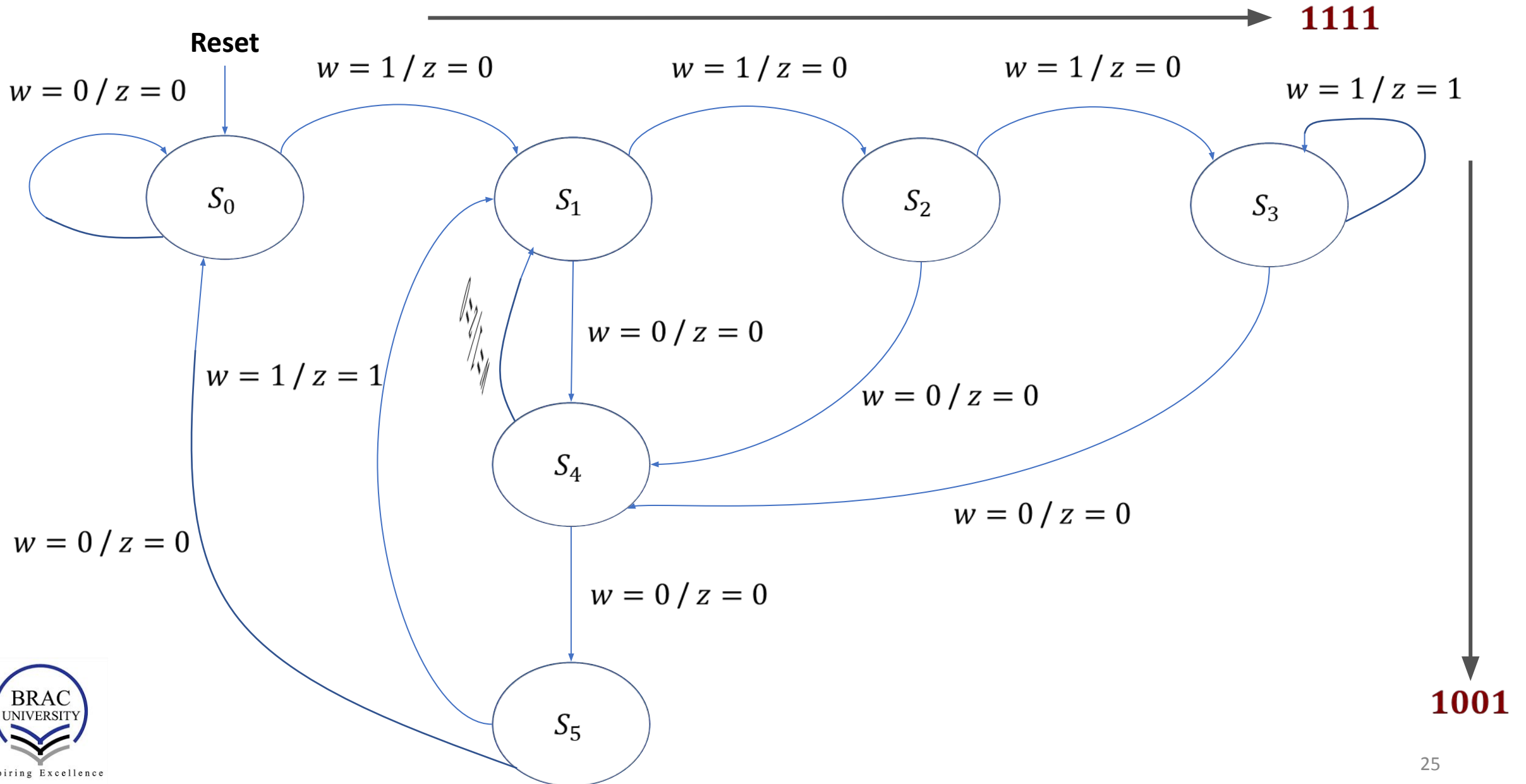
# An Example (Mealy type)

Problem: Derive the state diagram of the Mealy version for the example FSM that is, the FSM will generate  $z = 1$  at the instant the input values of  $w$  have the pattern 1111 or 1001. Overlapping of the input patterns are allowed.

| Clock cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|-------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|
| w           | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 1  | 1  | 0  | 0  | 1  | 1  | 1  | 1  |
| z           | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1  | 0  | 0  | 0  | 1  | 0  | 0  | 1  |



# State diagram(Mealy type)



# State table (Mealy type)

| Present State | Next State |     | Output z |     |
|---------------|------------|-----|----------|-----|
|               | w=0        | w=1 | w=0      | w=1 |
| S0            | S0         | S1  | 0        | 0   |
| S1            | S4         | S2  | 0        | 0   |
| S2            | S4         | S3  | 0        | 0   |
| S3            | S4         | S3  | 0        | 1   |
| S4            | S5         | S1  | 0        | 0   |
| S5            | S0         | S1  | 0        | 1   |

Figure : State Table

|         | Next State |         | Output z |     |
|---------|------------|---------|----------|-----|
|         | w=0        | w=1     | w=0      | w=1 |
| 000(S0) | 000(S0)    | 001(S1) | 0        | 0   |
| 001(S1) | 100(S4)    | 010(S2) | 0        | 0   |
| 010(S2) | 100(S4)    | 011(S3) | 0        | 0   |
| 011(S3) | 100(S4)    | 011(S3) | 0        | 1   |
| 100(S4) | 101(S5)    | 001(S1) | 0        | 0   |
| 101(S5) | 000(S0)    | 001(S1) | 0        | 1   |

Figure : State-assigned Table